

Understanding Mathematical Concepts via Go Language

Introduction to Go Language

Hyosang Kang (DGIST)

고 언어로 수학적 개념 이해하기

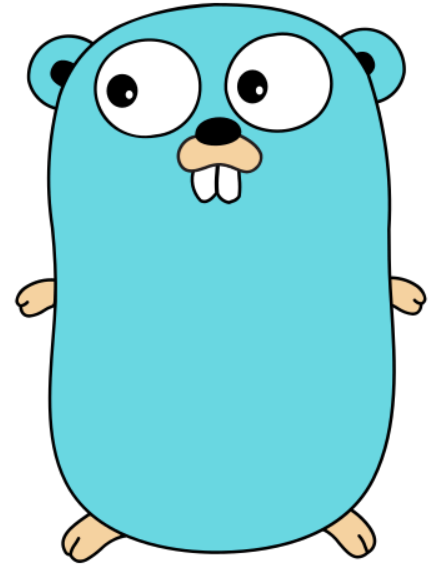
고 언어 소개

강효상 (DGIST)

Introduction to Go Language

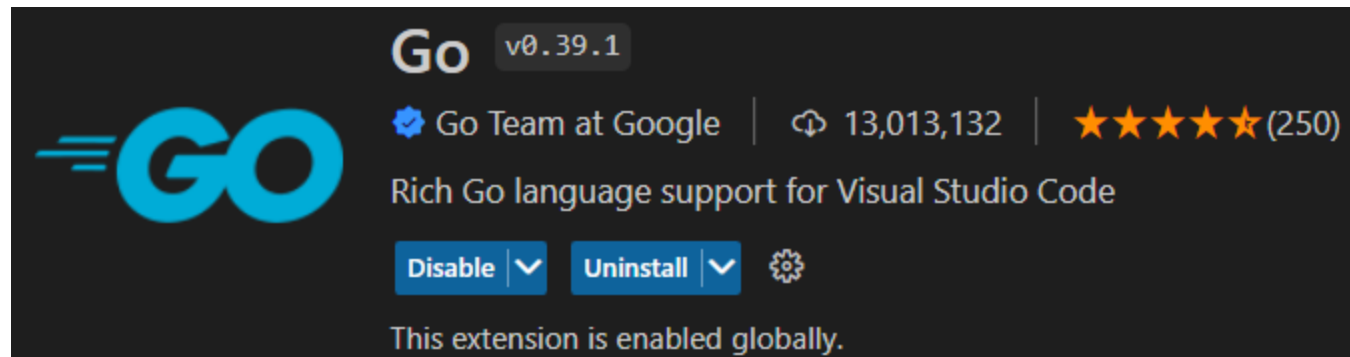
What is Go Language?

- Go is an open-source programming language developed by Google engineers Robert Griesemer, Rob Pike, and Ken Thompson and released in 2009.
- Go is designed for simplicity, reliability, and efficiency.
- Installation and documentations are available at <https://golang.org/>



Setting Up Go

- Install Visual Studio Code at <https://code.visualstudio.com>
- Install "Go" extension.



Your First Go Program

1. Create `main.go`.
2. Write the code.

```
package main

func main() {
    return
}
```

3. Run

```
> go run main.go
```

Module

- A **Go module** is a collection of related Go packages.

```
> go mod init gocalc
```

- A typical Go module looks like ...

(root_folder)/

├── go.mod (*contains module information*)

├── go.sum (*contains dependency information*)

├── (file1).go (*a go file for the root package*)

└── (sub_package)\ (**package**)

 └── (file2).go (*a go file for the sub package*)

Package

- A **package** in Go is a collection of Go files located in the same folder.
- The name (must be lower-cased) of the folder is the name of the package.
- The first line of every Go file declares the package it belongs to.

```
package gocalc
```


Main Package

- A Go file that declares the `main` package is compiled to an executable file.
- `main` package is not dependent to the folder name.
- It is desired to have only one Go file in the `main` package.

```
gocalc/  
├── go.mod  
├── go.sum  
├── ...  
└── main\  
    └── main.go
```

Basic Types in Go

- Boolean: `bool`
- Integer: `int`, `int8`, `int16`, `int32`, `int64`, `uint`, `uint8`, `uint16`, `uint32`, `uint64`
- Floating-point: `float32`, `float64`
- Complex: `complex64`, `complex128`
- String: `string`

More Types in Go

- **Composite Types**
 - **Array:** `[2]float64`
 - **Slice:** `[]int`
 - **Map:** `map[string]int`
- **Function Types:** `func(int) float64`
- **Pointer Types:** `*float64`

Struct types

- A **struct** is a collection of variables, called **fields**, that can be used as a single type.

```
type GraphOption struct {  
    Nsub                int  
    Xmin, Xmax, Ymin, Ymax float64  
    Width, Height      int  
    Filename            string  
}  
  
func (opt GraphOption) Pixel(x, y float64) (int, int) {  
    dw := (opt.Xmax - opt.Xmin) / float64(opt.Width)  
    dh := (opt.Ymax - opt.Ymin) / float64(opt.Height)  
    i := int((x - opt.Xmin) / dw)  
    j := int((opt.Ymax - y) / dh)  
    return i, j  
}
```

Interface Type

- An **interface** is a type defined by a set of **method**.
- A **method** is a function with a specific receiver argument.

```
type Additive interface {  
    Add(Additive) Additive  
    AddInv() Additive  
    Zero() Additive  
}
```

Type Parameters

```
type Element interface {  
    Equals(any) bool  
}  
  
type Set[T Element] interface {  
    Contains(T) bool  
}
```

```
func Max[T int | float64](x, y T) T {  
    if x < y {  
        return y  
    }  
    return x  
}
```

Embedding Interfaces

```
type Map[T1, T2 any] interface {  
    Map(T1) T2  
}  
  
type Finite interface {  
    Len() int  
}  
  
type Point interface {  
    Element  
    Finite  
    Map[int, Real]  
}
```

Implementing Interfaces

```
type SimpleVector []float64
```

- `SimpleVector` is defined in a different package.
- `SimpleVector` implements the `Point` interface.

```
func (p SimpleVector) Map(i int) gocalc.Real {  
    return SimpleReal(p[i])  
}  
  
func (p SimpleVector) Len() int {  
    return len(p)  
}
```



```

func (p SimpleVector) Equals(q any) bool {
    if qq, ok := q.(gocalc.Point); !ok {
        return false
    } else if len(p) != qq.Len() {
        return false
    } else {
        for i := 0; i < len(p); i++ {
            if !SimpleReal(p[i]).Equals(qq.Map(i)) {
                return false
            }
        }
        return true
    }
}

```

This video was produced with the support of CTL in DGIST.

Production: Hyosang Kang

Editing: Jongwoo Baek

본 동영상은 DGIST의 CTL의 지원을 받아 제작되었습니다.

제작: 강효상

편집: 백종우